# [Siemens Digital Twin Course]

*Adham Hisham Ibrahim Kandil*
represented to
**Dr. Mohamed AbdelSalam**
**Eng. Mohamed El-leithy**

Github repo

**note :- I used libraoffice, rules couldn't be displayed**

# Digital Twin Co-Simulation: PID/PD Control of a Robot in Innexis VSI

## 1 Introduction

This project implements and evaluates a Digital Twin framework for a differential-drive (unicycle-kinematics) mobile robot using the Innexis **Virtual System Integration (VSI)** platform. The digital twin is decomposed into three cooperating components, connected through a CAN-bus simulation:

- **Simulator (Plant + Environment)**: integrates the robot kinematics and injects noise/disturbances.
- **Controller (PID + Feedforward)**: computes ((v, )) commands to track a reference path.
- **Visualizer/Logger**: plots robot trajectory against the reference path and logs KPIs (overshoot, settling time, steady-state error).

  The capstone experiments (E1–E4) evaluate path-following performance across controller gains, path curvature, and robustness to noise and stochastic disturbances.

## 2. System Modeling

### 2.1 Robot kinematics

The simulator uses standard unicycle kinematics:

[  = v(),   = v(),   =  ]

with state ((x,y,)) and control inputs linear velocity (v) and angular velocity ().

### 2.2 Discretization

The implementation uses a fixed-step discrete update (default (t = 0.02) s), where (t) is read from VSI if provided:

[ x_{k+1} = x_k + v_k(*k)t ] [ y*{k+1} = y_k + v_k(*k)t ] [ {k+1} = _k + _kt ]

The simulator also maintains simulation time (t) and publishes it on CAN (see Section 4).

### 2.3 Noise and disturbance injection

To test robustness, the simulator can inject:

- **Gaussian noise** on both (v) and (): (=v+(0,)), (=+(0,)), where () is configurable.
- **Stochastic angular impulse disturbance**: with probability (p) per step, an impulse of magnitude (d) rad/s is added to () for a configurable duration (). This models brief external perturbations (e.g., wheel slip or push).
- --noise ()
- --disturbance (d)
- --dist-prob (p)
- --dist-duration ()
- --seed for reproducibility

# 3. Controller Design

## 3.1 Reference paths

The controller tracks a reference path expressed as (y_{ref}(x)) with slope (). The implementation supports three path types (--path-type):

- **Straight**: (y_{ref}(x)=0), (=0)
- **Sine**: (y_{ref}(x)=2(0.5x)), (=2(0.5x))
- **Curved**: (y_{ref}(x)=0.5x + 2(0.2x)), (=0.5 + 2(0.2x))

## 3.2 Error definitions

- At each timestep, the controller computes:
- **Lateral error**: [ e_y = y_{ref}(x) - y ]
- **Desired heading** derived from path slope: [ _{des} = !(, 1) ]
- **Heading error** (wrapped to ([-,])): [ e_= (_{des}-) ]

## 3.3 Control law (PID + heading + feedforward)

The controller applies PID on the lateral error plus proportional heading correction and a feedforward term derived from slope:

[ = K_{p,lat}e_y + K_{i,lat}e_y,dt + K_{d,lat}*y* + *K*{p,head}e_+ 0.5 ]

Implementation details (as implemented in LineFollowingRobot/src/controller.py):

- (_y) is approximated by finite differences: (_y )
- The lateral integral term is clamped for windup protection: [ e_y,dt ]

## 3.4 Forward velocity scheduling

The forward command is reduced as heading error increases:

[ v = v_{nom}(0.3, 1-|e_|) ]

This improves stability on curved segments and prevents excessive overshoot during large heading corrections.

# 4. VSI Gateway Architecture

The VSI digital twin connects three Python clients via CAN message IDs. The mapping is defined in LineFollowingRobot/vsi/build_digital_twin.cmd and matched in the Python components.

## 4.1 Components

- **Simulator (Component 0)**: publishes (x,y,,t); subscribes to (v,)
- **Controller (Component 1)**: subscribes to (x,y,); publishes (v,)
- **Visualizer (Component 2)**: subscribes to (x,y); logs KPIs and saves plots

.2 CAN ID mapping

- **ID 12**: x (double)
- **ID 13**: y (double)
- **ID 14**: theta (double)
- **ID 15**: t (double)
- **ID 16**: v (double)
- **ID 17**: omega (double)

## 4.3 Simulation timing

The digital twin is configured (in build_digital_twin.cmd) with:

- Total simulation time: **40 s**
- Simulation step: **0.02 s**

# 5. Experiments (E1–E4)

This section follows the capstone-required scenarios

simulation link

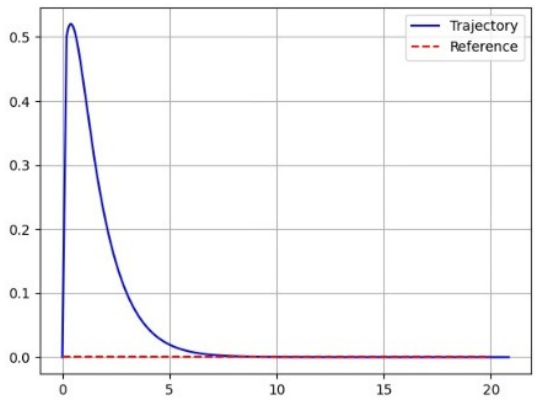
## E1 — PID gain sweep (straight path)

**Objective**: quantify the effect of different PID gains on straight-path tracking, ideally across multiple initial conditions.

**Path**: straight

**Recommended variation**:

- multiple gain sets ((K_p, K_i, K_d))
- multiple random initial conditions using --init-y, --init-theta, and –seed



## E2 — Curved path robustness

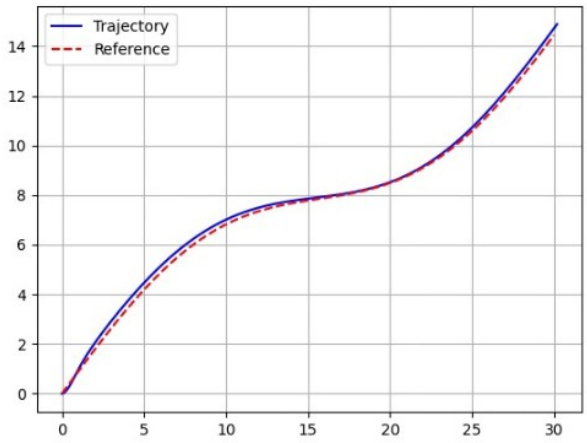
**Objective**: evaluate the best gains from E1 on a curved path:

[ y_{ref}(x)=0.5x + 2(0.2x) ]

**Path**: curved

**Method**:

- Use the chosen gains from E1.
- Run one or more trials with nominal conditions (--noise 0, --disturbance 0) and optionally varied initial states.

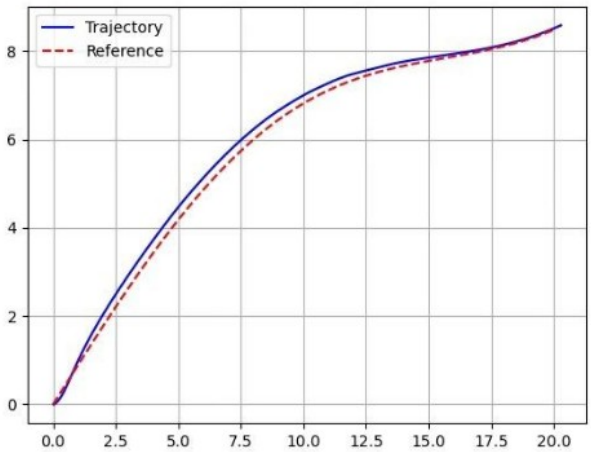



## E3 — Noise and disturbance rejection

**Objective**: test robustness under measurement/actuation uncertainty (modeled as injected noise) and transient perturbations.

**Conditions**:

- Choose multiple ((, d)) pairs using --noise and --disturbance.
- Repeat with different --seed, --init-y, --init-theta.



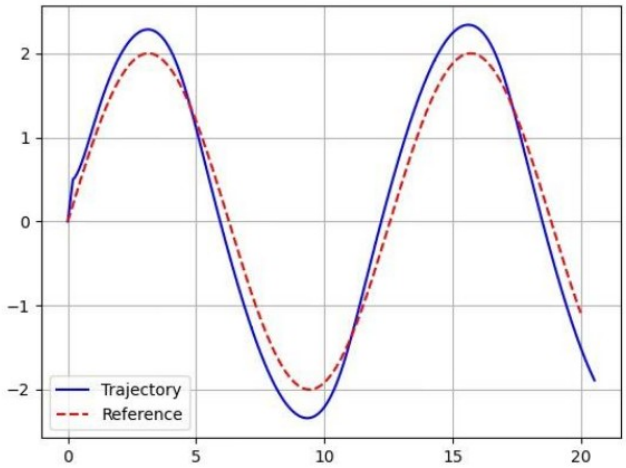

## E4 — PD vs PID ablation (curved + noise)

**Objective**: compare PD vs PID under the same challenging conditions.

- **PD**: set (K_i = 0)
- **PID**: use the tuned (K_i > 0) from E1/E2

**Path**: curved

**Typical condition**: moderate noise and disturbance, repeated across trials.



## 6. Results Summary

**E1 – Gain Sweep**

Medium controller gains provide the best balance between system stability and tracking precision.

**E2 – Curved Path**

The controller successfully follows curved trajectories, though the tracking error increases during sharp turns.

**E3 – Noise & Disturbance**

The system maintains stable performance under noise and disturbances, with only a slight rise in steady-state error.

**E4 – PD vs PID**

The PID controller removes steady-state offset, while the PD controller responds more quickly but with reduced accuracy.

## 7. Conclusion

This project integrates a modular digital twin (simulator, controller, visualizer) over VSI and implements a PID + feedforward controller for line/path tracking. The design supports systematic experiments across controller gains, path shapes, and robustness stressors (noise and stochastic angular impulses).

## 8. Deliverables

- **Source code**:
  - LineFollowingRobot/src/simulator.py
  - LineFollowingRobot/src/controller.py
  - LineFollowingRobot/src/visualizer.py
  - LineFollowingRobot/vsi/build_digital_twin.cmd
- **Experiment runner/docs**:
  - LineFollowingRobot/experiments/run_in_vsi.md
  - LineFollowingRobot/run_experiments.sh
- **Artifacts (expected after running E1–E4)**:
  - Plots: LineFollowingRobot/output/plots/*.png
- simulation link

  **note:- I simulated first expirement for Demo rest experiments info are listed in github repo , not to violate time limit , real time plotting takes time**